

COMP 3000 (WINTER 2023) OPERATING SYSTEMS

TUTORIAL 9 [\[LINK\]](#)

Tasks/Questions

The purpose of the following questions and tasks is to help you understand how containers work, which is an important artifact on top of and based on modern operating systems. The focus is on the OS building blocks – how it is made possible by the OS. Docker is just a typical and popular example to facilitate understanding.

You need to sudo for all the docker commands below.

Install Docker with `sudo apt install docker.io` if it's not already installed.

1. Use `docker images` and `docker ps` (similar to the purpose of `ps`) to see what images you have and whether any containers are running. Run your first container with `docker run hello-world`. Now, check the images and `docker ps` again, what do you see and why?
There was no image hello-world at the beginning. After running docker-run, you will see hello-world from docker-images (because it's downloaded from Docker Hub). But still we see nothing from docker-ps, as hello-world is short-running (finished quickly).
2. Look at the hello-world image above and you may wonder how the tiny-sized image forms a full container. Find out where the files are located (hint: using the command `docker inspect` and looking for paths starting with `/var/lib/docker`). What is the size of the found file? Run the file directly. What do you think is the way the tiny-sized image forms the full container?
You will see long pathnames one lower and one upper forming the layered file system. Directly running the small binary hello (found in one of the directories) will have the same effect as inside the container. The merged view of the layers will only be available inside the container.
This question may trick you into thinking there's something other than the small binary. Actually, it was statically linked (meaning it contains everything it needs) and is stand-alone.
3. `docker ps -a` will show something different. Next, let's try an image of a bit more complexity that will remain running. Try the centos (a Linux distribution) image: `docker run -d centos sleep infinity` (man `docker-run` for the meaning of the options).
docker ps -a also shows terminated containers, so you'll see hello-world. The next command is to run the centos image as a new container in the detached mode (without interacting with it via a terminal). A specific program sleep is run with the argument infinity (which will keep sleeping. See `man strtod`). This way, we can take our time to examine the running container.
4. Start a new shell: `docker exec -it <CONTAINER ID> /bin/bash` (you can find out the container ID with `docker ps -a`. Note that you can specify just the first few digits of the ID as long as it's distinctive). Explore the file system therein.
As opposed to -d, -it is to remain interactive (by redirecting the stdin to a pseudo-tty).
Next, we will examine the namespaces.

5. Compare the mount namespaces (different views of the file system structure) between the centos container and the host, using `ls -l /proc/self/ns/mnt`. Are they the same and if not, what are they (format: `mnt: [????]`)? In particular, take a look at the root mount point ("/) inside and outside the container (`mount | grep "on / "`). You can always use two sessions (one in the container and one in the host) or `tmux` as you feel convenient.

The root file system of the container will be the layered one (whose path you saw in `docker-inspect`). The namespaces will be different except for `cgroup` and `user`. The user namespace being shared also reflects the fact that with no UID mapping, the root in the container will be the root outside.

6. With the `lsns` command, you can get an overview of all namespaces. Comparing the centos container and the host, which namespaces are the same (shared) and which are different? **Here, you will have the same observations as above (all namespaces being different except for `cgroup` and `user`).**
7. Inside the container, use the command `id` to see what user you're logged in as (uid, gid and group). Then `ps -eo uid,gid,pid,command | grep sleep` inside the container and outside the container respectively. Are their outputs the same?

The user (UID) running `sleep` is the same inside and outside, which is both root (0).

8. Using the pid of "sleep" you have got outside the container above, find its pid namespace (`sudo ls -l /proc/<pid>/ns/pid`, format `pid: [????]`). Is it consistent with the `lsns` output inside the container?

This should be the same as what you saw with `lsns`.

9. From step 6, you have the individual namespaces used by the "sleep" container (no need to go inside the container again). Let's check the network namespace (`sudo nsenter -t <pid> -n ip addr`).

You can read the man page of `nsenter`, which enters the specified namespace (`-n` means the network namespace) of the target process and runs the specified command (here `ip addr`).

Note that uid remapping is not enabled by default. So the user (uid) inside the container will be the same outside (which does not consider security).

10. Now, make your own Docker container. Read the current [Dockerfile of hello-world](#). Change it in a way that it is no longer from scratch but *from hello-world*. Here we can avoid defining `csimpleshell` as the default command as it works with a terminal for interaction (then it will inherit `CMD` from `hello-world`). Build a new image called `csimpleshell` by doing the following:

```
mkdir anyname
```

```
copy your csimpleshell and the Dockerfile of hello-world into this directory
```

```
cd anyname
```

```
make necessary changes to the Dockerfile
```

```
docker build -t csimpleshell .
```

Here the changes you can consider can be to copy your `csimpleshell` binary into the container and base it on `hello-world`. Note that if you remove the `CMD` line it will just inherit from `hello-world` which can be overridden when you run it (see below).

So you can make it something like:

```
FROM hello-world
```

```
COPY csimpleshell /
```

11. Run the new csimpleshell container (e.g., `docker run -it csimpleshell /csimpleshell`). Depending on how you compiled csimpleshell, check whether the container runs or if it does whether csimpleshell works as before. Try compiling csimpleshell statically (if it was dynamic) and repeat the steps.
If your csimpleshell was dynamically linked, the error of file not found is expected, as the container image does not have the libraries. Even after you have it statically linked, inside the shell, still you cannot use any external commands (e.g., `ls`) as files are not there.
12. Edit the Dockerfile again and make it from “centos” instead of “hello-world”. Remove any reference to hello. Rebuild and run it. How is csimpleshell now? Why is there such a difference?
The new Dockerfile can be like:

```
FROM centos
COPY csimpleshell /
```

Now you should be able to do everything as before (issuing any commands like `ls`), as the centos image contains all the files needed.
Note that here it does not matter whether csimpleshell is statically linked or dynamically linked as the centos image includes everything. This is the benefit of a layered file system.